Reset Coursework

(CST2550 Software Engineering Management & Development)

Rajab Alasgarov (M01002269)

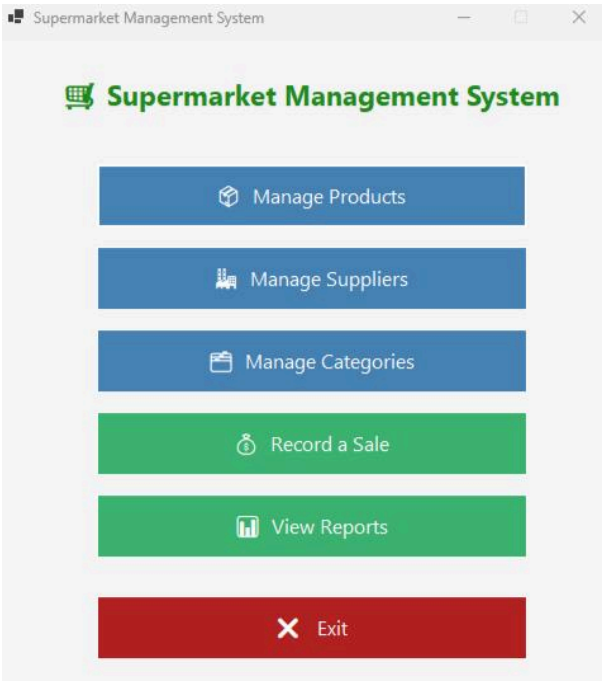
Table Of Contents

Introduction

Small supermarkets, or in other words local stores are mainly relying on a paper records system to manage their day-to-day operations, including stock records, product listings, supplier information and sales transactions. While manual methods may have been appropriate for small places in the past, they bring various risks as the business grows. Manual methods contain disadvantages like human error, are difficult to search quickly, vulnerable to physical damage or loss, and do not provide real time data like sales performance.

The Supermarket Management System was developed to address these challenges by providing a digital alternative for the paper-based records. System was built as Windows desktop application using C# .NET 10 and WinForms framework. The system provides an easy to use graphical interface for store employees with various levels of computer knowledge. Additionally, the system supports operations like product management, management of category and supplier, stock tracking with low stock alerts, sales recording and reports providing all the important data.

By replacing manual processes with such a system, the supermarket gains advantages of faster product searching, accurate stock counts, reliable sales records and more.



Data Structures & Algorithms

The system relies on two main strategies for searching and handling the product data. Primary data structure is a custom hash table created specifically for looking up products by barcode. When a product is inserted into the system, its barcode is passed through a hash function which multiplies each character value by 31 and divides by the table size to give an index. That position is then stored in a fixed array of 200 slots . When a barcode search is made, and the product is taken directly from its slot without searching each record in the database, the same function is executed again. This results in an average search period of O(1), and the speed is not dependent on the number of products stored.

If two separate barcodes belong to the same index we resolve collisions by linear probing. The algorithm moves through the array one slot at a time until it finds an empty position. A separate boolean array keeps track of which slots have been deleted so that the probing chain is not broken when a product is deleted. When a product screen opens, the hash table is loaded from the database and kept in sync with the database as products are added or removed.

A different approach is used for name searching of a product. Hash table is not an option because names are not unique and staff can search on part of a word, like searching for “milk” returns both “whole milk” and “skimmed milk”. Instead the system performs a linear search on all product records in the database and checks whether each title contains the searched term. This is O(n) time where “n” is the total number of products, but for the size of a small supermarket this is totally fine and the results come back instantly.

The database consists of 5 related tables: Products, Categories, Suppliers, Sales, and SaleItems. Products have foreign keys to both Categories and Suppliers. This means the database will not accept invalid data and data remains consistent. SaleItems assign specific products to completed sales transactions and also store the quantity sold and the sale price. The tables are processed and updated using C# objects with Entity Framework Core, so that you do not have to write raw SQL for most operations.

Time Complexity Analysis

One of the important factors to consider when designing such a system is how well it scales with the amount of data. This is written as Big O notation. O(1) means an operation that takes the same amount of time regardless of how many products there are. O(n) means an operation that takes longer the more products there are .

The quickest operation in the system is to query a product by barcode. It uses a custom hash table . So it calculates exactly where the product is stored and goes there directly . It does not matter whether there are 10 products or 10,000 as the lookup takes the same amount of time. This is O(1) on average.

Searching by product name is different. Staff may enter a partial word such as “milk” in order to identify several matching products, so the system must check each product in the database individually. More products. More time. This is O(n) , but for a small supermarket with a limited amount of products it is fast enough to be unnoticeable .

O(1) average case for adding a new product as well. The system validates that the title is not empty and that the price is valid and the barcode is not already taken. Then it saves the product to the database and adds it to the hash table. Each of these steps takes a constant amount of time.

The time required to record sales is O(n), with n denoting the total number of items in the basket. First, the system tests the availability of stock for all items, afterwards it once again scans the basket to lower the level of the stock. Since two scans of the basket are performed, the time required will depend on the number of items sold.

The generation of the low stock report is also an operation taking O(n) time. This is because the system scans all products available in the database to test whether they have gone below their particular low stock thresholds. Therefore, the time required for the process will increase with the number of products available in the database.

Operation	Time Complexity	Definition
Add Product	O(1) Average	Same speed regardless of product count.
Barcode Search	O(1) Average	Goes directly to the product position.
Name Search	O(n)	Check every product one by one.
Record Sale	O(n)	Checks and updates each basket item.
Low Stock Report	O(n)	Check every product against the threshold.

Testing

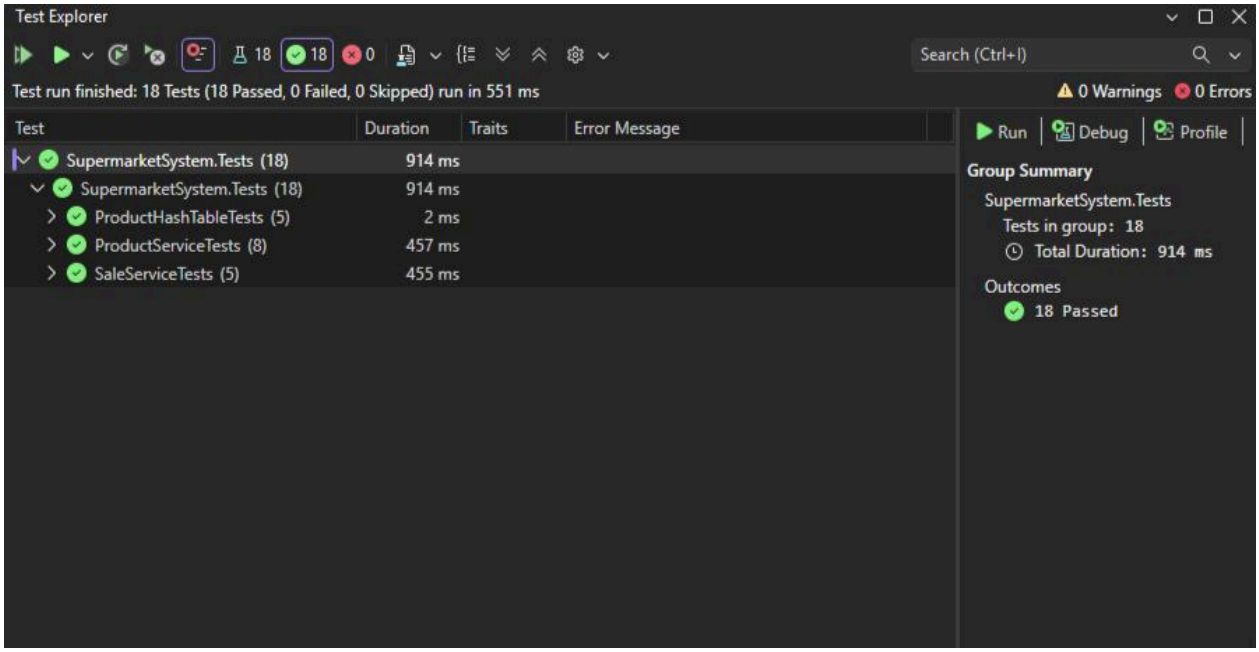
In order to make sure the system is working properly 18 automated tests were written which test the core features of the application. Each test creates a small scenario, runs an operation and verifies that the result was as expected. If the result is wrong, the test fails and indicates the problem. For making tests quick and simple, a temporary database that exists only in memory was used instead of connecting to the real SQL Server database. This means that the tests have no impact on any real data and can be run in seconds. Each test begins with a new database, ensuring that no test influences the results of another.

The barcode lookup system is checked by five tests. They show that a product can be stored and retrieved through the barcode, that searching for a faulty barcode displays no results, that deleting a product makes it unsearchable, and that the system correctly handles storage and retrieval of multiple products.

There are 8 tests to verify the features of product management. These show that a valid product is saved correctly, that two products with the same barcode is prohibited, that a product with negative price or empty name is rejected, that name searches return the correct information, that barcode searches find the proper item, that the low stock feature flags only products that are actually low, and that deleting a product removes it completely.

Five tests look at the sales process. They ensure that when completing a sale an accurate total amount is saved, that the stock level is reduced by the correct amount after a sale, that it is not possible to complete a sale with no items, also that it is not possible to sell more than what is in stock and that the sales history is displayed with the most recent sale first.

All 18 tests passed successfully.



Conclusion

The Local Supermarket Management System fulfills the requirements outlined in the coursework brief providing a full digital alternative to the paper-based procedures provided in the scenario. Staff can manage products, categories, suppliers and record sales with automatic stock deduction, monitor low stock levels and review built-in reports. The system features a Windows desktop interface built in C# .NET 10 WinForms.

There are some limitations to the system which are important to mention. It runs on one machine, not allowing for multiple checkouts or shared access over the network, which would limit its usage in a larger store. It doesn't have user logins or role-based access control, so anyone from staff can delete products or view sales reports. The hash table is fixed at 200 slots. If the product catalogue grows large and the hash table is not resized, this could be a performance issue. Also, barcodes must be manually typed in instead of scanned, which would slow down real-life sales processing.

Some of the future improvements could be printing receipts for customers, support for barcode scanners, customer loyalty accounts, sales trend charts, multiple user network access and a login system to limit access based on roles of the staff. Dynamic resizing of hash tables would improve performance in the long run as the range of products expands. Lastly, there is always room for improvements, but the current system would perfectly fit a small supermarket and would really boost their performance.

References

Cataldo, A. (2025) Supermarket system: why is it essential? SYDLE Blog. Available at: https://www.sydle.com/blog/supermarket-system-68c423c3392e4e04c593466a (Accessed: 22 June 2026).

Laudon, K.C. and Laudon, J.P. (2020) Management information systems: managing the digital firm. 16th edn. Harlow: Pearson Education.

Microsoft (no date) Entity Framework Core documentation. Microsoft Learn. Available at: https://learn.microsoft.com/en-us/ef/core/ (Accessed: 24 June 2026).

Microsoft (no date) Windows Forms for .NET documentation. Microsoft Learn. Available at: https://learn.microsoft.com/en-us/dotnet/desktop/winforms/ (Accessed: 24 June 2026).

My Coding Project (2025) C# full project | supermarket management system [Online video]. Available at: https://www.youtube.com/watch?v=mEfTCGbTkHs (Accessed: 25 June 2026).